COSC2637/2633 Big Data Processing
Lab 3: Tutorial/Lab class (week 4)

Task 1 – Execute python-based MapReduce program

Recall "hadoop-streaming.jar" is used in the previous lab for executing the python-based MapReduce program. Today, we learn details of hadoop-streaming.jar. To learn more, you can visit https://hadoop.apache.org/docs/r1.2.1/streaming.html

Hadoop streaming is a utility that comes with the Hadoop distribution. The utility allows you to create and run Map/Reduce jobs with any executable or script as the mapper and/or the reducer. For example:

```
#hadoop jar /usr/lib/hadoop/hadoop-streaming.jar \
    -D mapred.reduce.tasks=3 \
    -file ./mapper.py \
    -mapper ./mapper.py \
    -file ./reducer.py \
    -reducer ./reducer.py \
    -input /3littlepigs.txt \
    -output /output
```

In the above example, both the mapper and the reducer are executables that read the input from stdin (line by line) and emit the output to stdout. The utility will create a Map/Reduce job, submit the job to an appropriate cluster, and monitor the progress of the job until it completes.

Mapper - When an executable is specified for mappers, each mapper task will launch the executable as a separate process when the mapper is initialized. As the mapper task runs, it converts its inputs into lines and feed the lines to the stdin of the process. In the meantime, the mapper collects the line-oriented outputs from the stdout of the process and converts each line into a key/value pair, which is collected as the output of the mapper. By default, the prefix of a line up to the first tab character is the key and the rest of the line (excluding the tab character) will be the value. If there is no tab character in the line, then entire line is considered as key, and the value is null. However, this can be customized, as discussed later.

Reducer - When an executable is specified for reducers, each reducer task will launch the executable as a separate process then the reducer is initialized. As the reducer task runs, it converts its input key/values pairs into lines and feeds the lines to the stdin of the process. In the meantime, the reducer collects the line-oriented outputs from the stdout of the process, converts each line into a key/value pair, which is collected as the output of the reducer. By default, the prefix of a line up to the first tab character is the key and the rest of the line (excluding the tab character) is the value. However, this can be customized, as discussed later.

Specifying the Number of Reducers

To specify the number of reducers, for example three, use:

```
#hadoop jar /usr/lib/hadoop/hadoop-streaming.jar \
    -D mapred.reduce.tasks=3 \
    -file ./mapper.py \
    -mapper ./mapper.py \
    -file ./reducer.py \
    -reducer ./reducer.py \
    -input /3littlepigs.txt \
    -output /output
```

Often, you may want to process input data using a map function only. To do this, simply set mapred.reduce.tasks to zero. The Map/Reduce framework will not create any reducer tasks. Rather, the outputs of the mapper tasks will be the final output of the job.

```
    -D mapred.reduce.tasks=0
```

Customizing How Lines are Split into Key/Value Pairs

As noted earlier, when the Map/Reduce framework reads a line from the stdout of the mapper, it splits the line into a key/value pair. By default, the prefix of the line up to the first tab character is the key and the rest of the line (excluding the tab character) is the value.

However, you can customize this default. You can specify a field separator other than the tab character (the default), and you can specify the nth (n >= 1) character rather than the first character in a line (the default) as the separator between the key and value. For example:

```
#hadoop jar /usr/lib/hadoop/hadoop-streaming-3.1.4.jar \
    -D stream.map.output.field.separator=. \
    -D stream.num.map.output.key.fields=4 \
    -D mapred.reduce.tasks=3 \
    -file ./mapper.py \
    -mapper ./mapper.py \
    -file ./reducer.py \
    -reducer ./reducer.py \
    -input /3littlepigs.txt \
    -output /output
```

In the above example, "-D stream.map.output.field.separator=." specifies "." as the field separator for the map outputs, and the prefix up to the fourth "." in a line will be the key and the rest of the line (excluding the fourth ".") will be the value. If a line has less than four "."s, then the whole line will be the key and the value will be an empty Text object (like the one created by new Text("")).

Similarly, you can use "-D stream.reduce.output.field.separator=SEP" and "-D stream.num.reduce.output.fields=NUM" to specify the nth field separator in a line of the reduce outputs as the separator between the key and the value.

Similarly, you can specify "stream.map.input.field.separator" and "stream.reduce.input.field.separator" as the input separator for Map/Reduce inputs. By default the separator is the tab character.

Hadoop Partitioner Class

Hadoop has a library class, KeyFieldBasedPartitioner, that is useful for many applications. This class allows the Map/Reduce framework to partition the map outputs based on certain key fields, not the whole keys. For example:

```
#hadoop jar /usr/lib/hadoop/hadoop-streaming-3.1.4.jar \
          -D stream.map.output.field.separator=. \
          -D stream.num.map.output.key.fields=4 \
          -D map.output.key.field.separator=. \
          -D mapred.text.key.partitioner.options=-k1,2 \
          -D mapred.reduce.tasks=3 \
          -file ./mapper.py \
          -mapper ./mapper.py \
          -file ./reducer.py \
          -reducer ./reducer.py \
          -input /3littlepigs.txt \
          -output /output \
        -partitioner org.apache.hadoop.mapred.lib.KeyFieldBasedPartitioner
```

Here, *-D stream.map.output.field.separator=.* and *-D stream.num.map.output.key.fields=4* are as explained in previous example. The two variables are used by streaming to identify the key/value pair of mapper.

The map output keys of the above Map/Reduce job normally have four fields separated by ".". However, the Map/Reduce framework will partition the map outputs by the first two fields of the keys using the *-D mapred.text.key.partitioner.options=-k1,2* option. Here, *-D map.output.key.field.separator=.* specifies the separator for the partition. This guarantees that all the key/value pairs with the same first two fields in the keys will be partitioned into the same reducer.

This is effectively equivalent to specifying the first two fields as the primary key and the next two fields as the secondary. The primary key is used for partitioning, and the combination of the primary and secondary keys is used for sorting.

Exercise-1. Customizing How Lines are Split into Key/Value Pairs

Suppose you process an employee data stored in file highestSalary_sample.txt.

ID	*NAME*	*AGE*	*GENDER*	*SALARY*
1201	*gopal*	*45*	*Male*	*50000*
1202	*manisha*	*40*	*Female*	*51000*

…

Write a python MapReduce program to count the number of employees based on their salary category and gender. The salary category is defined as:

salary<10000	*low*
10000<=salary<20000	*low_middle*
20000<=salary<30000	*high_middle*
30000<=salary	*high*

The output should be something like:

high_middle	*Male*	*4*
high	*Female*	*2*
high	*Male*	*4*
low_middle	*Female*	*2*
low	*Female*	*1*

The "mapper.py" and "reducer.py" has been provided in course week-4 module on Canvas. Read the code to understand how to develop a python MapReduce program. More importantly, "*-D stream.num.map.output.key.fields=2" is included in "run keys.sh" which specifies the input of reducer has the first two items as the key and the rest is the value.*

The complete run keys.sh file is below

runArgument.sh run_keys.sh

```
#!/bin/bash
hadoop jar /usr/lib/hadoop/hadoop-streaming.jar \
-D stream.num.map.output.key.fields=2 \
-D mapred.reduce.tasks=3 \
-file ./mapper.py \
-mapper ./mapper.py \
-file ./reducer.py \
-reducer ./reducer.py \
-input /highestSalary_sample.txt \
-output /output
```

The mapper.py is below

```
#!/usr/bin/env python3
import sys
# count the number of employee based on salary group
# salary<10000 low;
# 10000<=salary<20000 low middle;
# 20000<=salary<30000 high middle;
# 30000<=salary high;

# input comes from standard input STDIN
for line in sys.stdin:
    line = line.strip() #remove leading and trailing whitespaces
    id, name, age, gender, salary = line.split("\t") #split the line into words
    and returns as a list

    if float(salary)<10000: salaryC = "low"
    elif 10000<=float(salary)<20000: salaryC = "low_middle"
    elif 20000<=float(salary)<30000:salaryC = "high_middle"
    else: salaryC = "high"

    print('%s\t%s\t%s' % (salaryC,gender,"1")) #Emit <key, value> pairs
```

The reducer.py is below

```
#!/usr/bin/env python3
import sys
# Count the number of "salaryC gender"
# That is, "salaryC gender" is the key.

# using a dictionary to map words to their counts
current_salaryC = None
current_gender = None
current_count = 0
salaryC = None
gender = None

# input comes from STDIN
for line in sys.stdin:
    line = line.strip()
    salaryC,gender,count = line.split('\t')
    try:
        count = int(count)
    except ValueError:
        continue
    if current_salaryC == salaryC and current_gender == gender:
        current_count += count
    else:
        if current_salaryC:
            print('%s\t%s\t%s' % (current_salaryC, current_gender, current_count))
        current_count = count
        current_salaryC = salaryC
        current_gender = gender
if current_salaryC == salaryC and current_gender == gender:
    print('%s\t%s\t%s' % (current_salaryC, current_gender, current_count))
```

As instructed in previous lab, upload to AWS EMR cluster master node the following files

mapper.py
reducer.py
run_keys.sh
highestSalary_sample.txt

```
[hadoop@ip-192-168-26-253 ~]$ ls
hadoop-streaming-3.1.4.jar  mapper.py   run_keys.sh
highestSalary_sample.txt    reducer.py
```

Copy highestSalary sample.txt to HDFS. Execute run keys.sh.

If succeed, you will check the output of each reducer.

```
[hadoop@ip-192-168-22-131 ~]$ hadoop fs -cat /output/part-00000
[hadoop@ip-192-168-22-131 ~]$ hadoop fs -cat /output/part-00001
low_middle      Female  2
[hadoop@ip-192-168-22-131 ~]$ hadoop fs -cat /output/part-00002
high    Female  2
high    Male    4
high_middle     Male    4
low     Female  1
```

Exercise-2. *Hadoop Partitioner Class*

So far, Exercise-1 cannot ensure all male-related records are processed and output by the same reducer, so are all female-related records. To make it, we can use "KeyFieldBasedPartitioner". *While all same, the run_partitioner.sh will be used.*

runArgument.sh | run_keys.sh | run_partitioner.sh

```
1  #!/bin/bash
2  hadoop jar /usr/lib/hadoop/hadoop-streaming.jar \
3  -D stream.num.map.output.key.fields=2 \
4  -D mapred.text.key.partitioner.options=-k2,2 \
5  -D mapred.reduce.tasks=3 \
6  -file ./mapper.py \
7  -mapper ./mapper.py \
8  -file ./reducer.py \
9  -reducer ./reducer.py \
10 -input /highestSalary_sample.txt \
11 -output /output \
12 -partitioner org.apache.hadoop.mapred.lib.KeyFieldBasedPartitioner
```

After execution, you can see the output from each reducer.

```
[hadoop@ip-192-168-22-131 ~]$ hadoop fs -cat /output/part-00000
[hadoop@ip-192-168-22-131 ~]$ hadoop fs -cat /output/part-00001
high    Female  2
low     Female  1
low_middle      Female  2
[hadoop@ip-192-168-22-131 ~]$ hadoop fs -cat /output/part-00002
high    Male    4
high_middle     Male    4
```

Hadoop Comparator Class

Hadoop has a library class, KeyFieldBasedComparator, that is useful for many applications. This class provides a subset of features provided by the Unix/GNU Sort. For example:

run_partitioner_sort.sh

```
#!/bin/bash
hadoop jar /usr/lib/hadoop/hadoop-streaming.jar \
-D mapred.output.key.comparator.class=org.apache.hadoop.mapred.lib.KeyFieldBasedComparator \
-D stream.num.map.output.key.fields=2 \
-D mapred.text.key.partitioner.options=-k1,1 \
-D mapred.text.key.comparator.options='-k2,2nr -k1,1' \
-D mapred.reduce.tasks=3 \
-file ./mapper.py \
-mapper ./mapper.py \
-file ./reducer.py \
-reducer ./reducer.py \
-input /highestSalary_sample.txt \
-output /output \
-partitioner org.apache.hadoop.mapred.lib.KeyFieldBasedPartitioner
```

The mapper output keys of the above Map/Reduce job normally have four fields separated by ".". The mapper output is partitioned by the first key, and each partition will be sorted, before processed by the reducer, by the second field and then by the first field of the keys using the

-D mapred.text.key.comparator.options='-k2,2nr -k1,1'

Here, *-n* specifies that the sorting is numerical sorting and *-r* specifies that the result should be reversed.

Task 2 – How to develop python-based MapReduce program

When you develop a python MapReduce program, it is helpful to take two stages.

Stage-1. Test your code (cat data | map | sort | reduce) on local machine (Linux)

cat highestSalary_sample.txt | python mapper.py | sort -k1,1 -k2,2 | python reducer.py

```
[hadoop@ip-192-168-22-131 ~]$ cat highestSalary_sample.txt | python mapper.py |
sort -k1,1 -k2,2 | python reducer.py
high    Female  2
high    Male    4
high_middle     Male    4
low     Female  1
low_middle      Female  2
```

"sort -k1,1, -k2,2" sort all lines, comparing the contents of field 1, and when two lines have the same content in field 1, comparing the contents of field 2.

You can also test step by step to identify bugs in your code.
cat highestSalary_sample.txt
cat highestSalary_sample.txt | python mapper.py
cat highestSalary_sample.txt | python mapper.py | sort -k1,1 -k2,2
cat highestSalary_sample.txt | python mapper.py | sort -k1,1 -k2,2 | python reducer.py

Stage-2. Test your code on AWS EMR cluster

Note the code running successfully in stage-1 may report error when running on AWS EMR cluster. To debug in this situation, you can

1. set "-D mapred.reduce.tasks=0" to check whether the output of mapper is correct or not
2. comment the code in reducer.py and uncomment them block by block; and print the relevant information. The printed information can be found in the output of reducer.